

Lab Manual

Subject: -Operating System-Lab (UNCSPR304)
B.Tech (Semester-5th)



Department of Computer Science & Engineering

G H RAISONNI UNIVERSITY, AMRAVATI

SCHOOL OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

NAME OF PROGRAM: B.TECH

PRACTICAL LIST

Name of Course:	Operating System -Lab	Course Code:	UNCSPR304
Year/Semester:	B.Tech Sem-V	Session:	2025-26

Course Outcome: After completion of the course, students will be able to

CO1: Identify basic structure and purpose of operating system

CO2: Interpret the concepts of process and illustrate various CPU scheduling algorithms

CO3: Interpret the concepts of inter process communication.

CO4: Schematize Deadlock & File System security mechanisms in operating systems.

CO5: Analyze different memory management techniques with advantages and disadvantages.

Sr. No.	Name of Experiments	CO
1	Perform a case study by installing and exploring various types of operating systems on a physical or logical (virtual) machine. (Linux Installation).	CO1
2	Implement of different types of system calls.	CO1
3	Write a program to implement FCFS, SJF and priority based Algorithm.	CO2
4	Write a program to implement Round Robin Algorithm.	CO2
5	Write a program to implement Producer Consumer Problem.	CO3
6	Write a program to implement Dining-Philosophers Problem.	CO3
7	Write a program to implement Bankers algorithm.	CO4
8.	Write a program to implement Page Replacement.	CO4
9.	Write a program to implement Memory management algorithm	CO5
10.	Write a program to implement Disk Scheduling algorithms.	CO5
Open Ended Practical		
11.	Write C programs to simulate Intra & Inter – Process Communication (IPC) techniques: Pipes, Messages, Queues and Shared Memory.	
12.	Write C program to Perform Shell Programming.	

Experiment No.01

Name of Experiment:

Perform a case study by installing and exploring various types of operating systems on a physical or logical (virtual) machine. (Linux Installation).

Problem Statement: Installation of Fedora 42 Workstation on physical Machine

Software /Hardware Requirements: Fedora 42 Workstation ISO Image Pen Drive (Bootable) ,PC

Theory:

1. Introduction to Linux

Linux is a powerful, open-source, UNIX-like operating system originally developed by **Linus Torvalds** in 1991. It has since grown into one of the most widely used OS kernels, forming the basis for various **distributions (distros)** like Ubuntu, Fedora, Debian, Arch, Red Hat, etc.

Linux is widely used in:

- Servers
- Desktops
- Embedded Systems
- Mobile Devices (e.g., Android)
- Supercomputers
- IoT Devices

2. Key Features of Linux

Feature	Description
Open Source	Source code is freely available and modifiable
Multiuser System	Multiple users can use system resources simultaneously
Multitasking	Can run multiple processes concurrently
Portability	Can run on a wide range of hardware platforms
Security	Offers user permissions, firewalls, SELinux, etc.
Stability	Rarely needs to be rebooted; ideal for servers
Modularity	Components like kernel, shell, filesystem are independent
Shell Interface	Command-line interface to control the system

3. Components of Linux Operating System

A. Kernel

- Core component managing hardware and system resources
- Types: Monolithic (Linux), Microkernel, Hybrid
- Responsibilities: Process scheduling, memory management, device drivers

B. Shell

- Interface between user and kernel
- Examples: Bash (default), Zsh, Fish
- Interprets commands entered by users

C. File System

- Everything in Linux is treated as a file
- Standard hierarchy starts from / (root)
- Common directories: /home, /etc, /bin, /var, /dev

D. System Utilities

- Essential tools and programs for managing the system
- Example: ls, cd, cp, ps, top, kill

E. Application Programs

- Graphical and text-based software like browsers, editors, IDEs

4. Linux Distributions (Distros)

A Linux **distribution** is a complete operating system built around the Linux kernel, with added tools and package managers.

Distribution	Key Features
Ubuntu	User-friendly, great for beginners
Fedora	Cutting-edge features, Red Hat-backed
Debian	Very stable and community-driven
Arch	Lightweight, rolling release, advanced users
CentOS/AlmaLinux	Server-based, enterprise-ready

5. Linux File System Hierarchy

Linux follows a standard file system layout:

/	
— bin/	→ Essential binaries
— boot/	→ Bootloader files
— etc/	→ Configuration files
— home/	→ User home directories
— root/	→ Root user's home
— usr/	→ User programs and data
— var/	→ Variable data like logs, mail

6. Common Linux Commands

Command	Purpose
ls	List files/directories
cd	Change directory
pwd	Print current directory
cp, mv	Copy or move files
rm	Remove files
chmod	Change file permissions
top, htop	Show running processes
sudo	Run command with super user privileges
dnf, apt	Install and manage packages

7. Package Management

- Linux uses **package managers** to install, update, and remove software.
- Common Package Managers:
 - **APT**: Debian-based (Ubuntu)
 - **DNF/YUM**: Red Hat-based (Fedora, CentOS)
 - **Pacman**: Arch Linux
 - **Flatpak/Snap**: Universal formats for all distros

8. Applications of Linux

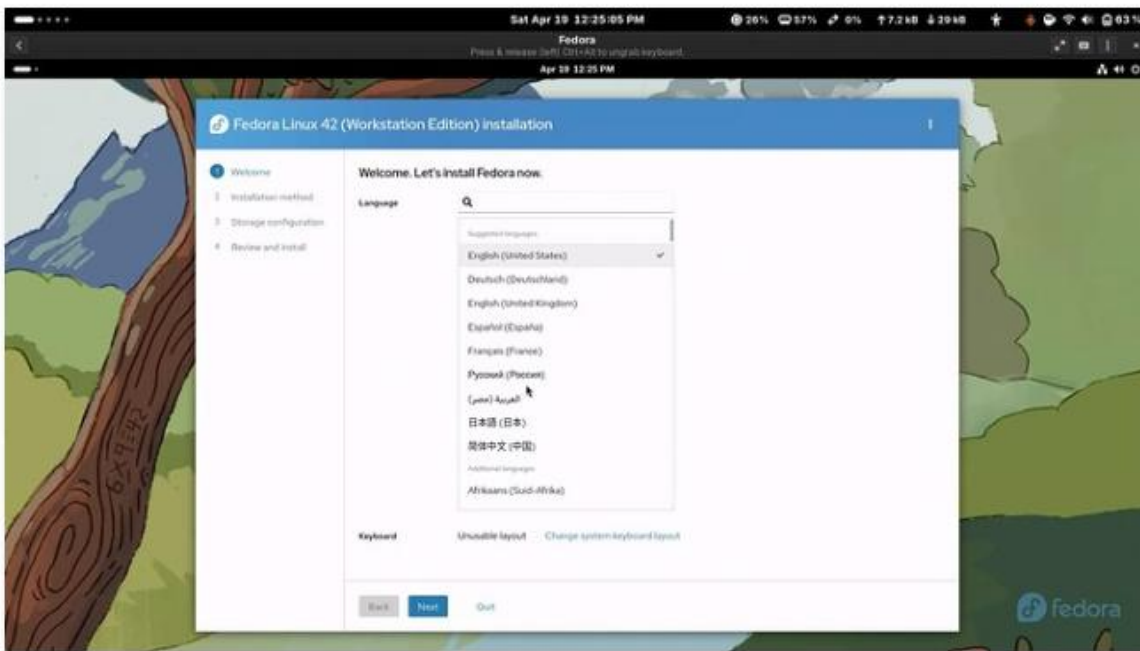
- **Servers**: Web, file, DNS, email servers
- **Software Development**: Supports a wide range of programming languages and tools
- **Networking**: Routing, switching, network analysis
- **Cybersecurity**: Tools like Kali Linux for penetration testing
- **Education and Research**
- **Cloud Platforms**: AWS, Azure use Linux extensively

9. Advantages of Linux

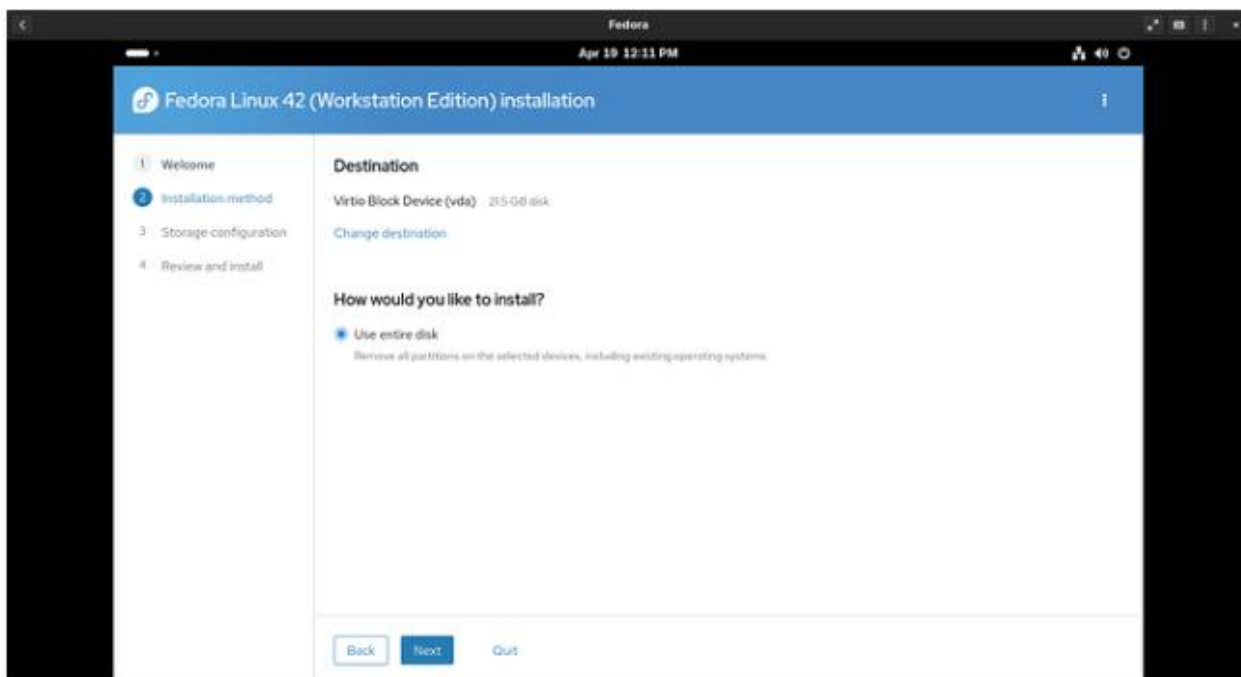
- Free and open-source
- Highly customizable
- Secure and stable
- Great community support
- Large software repositories

Fedora 42 Workstation Installation Process:

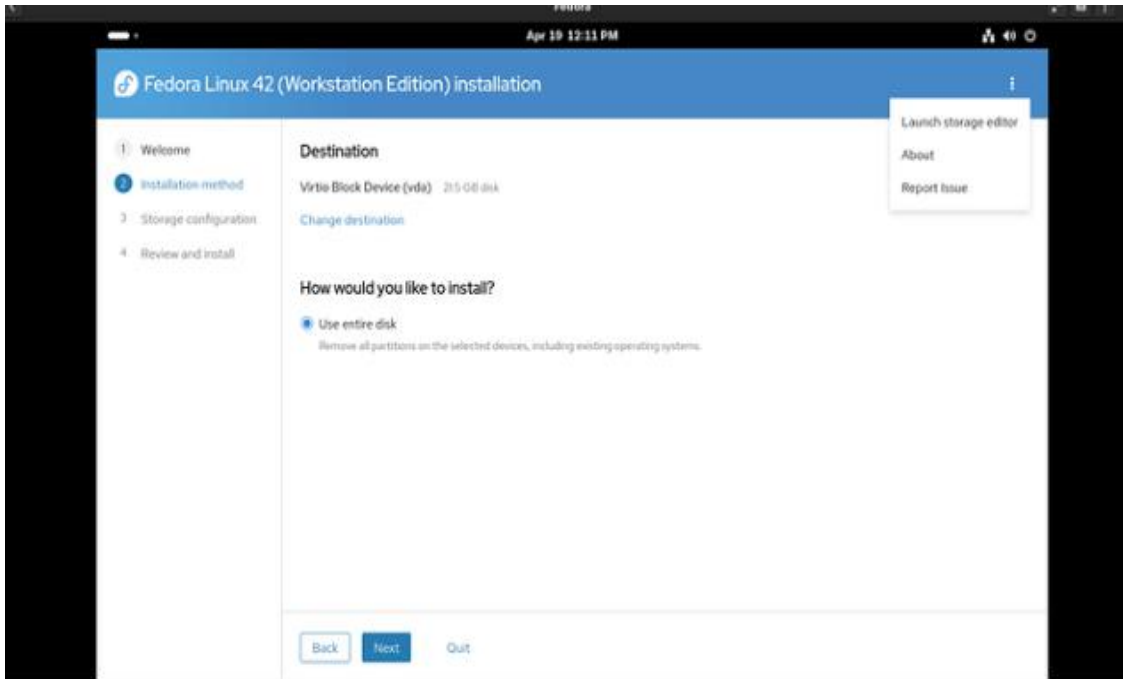
1. Boot into fedora 42 using live iso using your preferred way of flashing the iso
2. Start the installer and select you language and keyboard layout



3. Here you will be listed with available partition, go to the top right 3 dot



4. From the 3 dot select **Launch storage editor**



5. Create the Free Space on the Disk by Shrink the Size
6. Linux required 40 GB of Free Space
7. Install the Setup and finished the installation

To dual-boot Fedora 42 and Windows, you'll need to prepare your system by creating free space for Fedora, creating a bootable USB drive with Fedora 42, and then installing Fedora, making sure to choose the correct partitioning scheme during installation.

Here's a more detailed breakdown:

1. Preparation:

- **Backup:**

Back up your important data from your Windows system, as partitioning can be risky.

- **Shrink Windows Partition:**

In Windows, use Disk Management to shrink the existing Windows partition and create unallocated space for Fedora. You'll need at least 40GB for Fedora.

- **Download Fedora:**

Download the Fedora Workstation 42 ISO file from the Fedora Project website.

- **Create Bootable USB:**

Use a tool like Rufus (or Fedora Media Writer, which is recommended) to create a bootable USB drive from the ISO file. Ensure the partition scheme is set to GPT.

2. Installation:

- **Boot from USB:** Boot your computer from the Fedora bootable USB drive.
- **Start Installation:** Launch the Fedora installer and choose your language and keyboard layout.

- **Choose Installation Destination:** Select the unallocated space you created earlier as the installation destination. You can choose the automatic partitioning option, or you can create custom partitions if you prefer.
- **Partitioning (if custom):** If you choose custom partitioning, you'll need to create at least three partitions: /boot/efi (for the UEFI bootloader), /boot (for the kernel and other boot files), and / (the root partition). You might also want to create a /home partition for your user files.
- **Encryption (optional):** You can choose to encrypt your Fedora installation for added security.
- **Review and Install:** Review your installation settings and click "Begin Installation".
- **Final Setup:** Once the installation is complete, you'll be guided through the final setup steps.

3. Dual-Boot Configuration:

- **GRUB Bootloader:**

Fedora's installer will configure the GRUB bootloader to allow you to choose between Windows and Fedora at start up.

- **BIOS/UEFI settings:**

In some cases, you may need to enter your computer's BIOS/UEFI settings and adjust the boot order to prioritize GRUB or Fedora if it doesn't automatically boot into the GRUB menu.

Important Considerations:

- **Secure Boot:**

If your computer has secure boot enabled, you might encounter issues during installation or booting. You may need to disable secure boot in your BIOS/UEFI settings, or ensure Fedora is properly signed to work with secure boot.

- **Disk Partitioning:**

Carefully review the partitioning steps to avoid accidentally deleting your Windows installation.

- **Troubleshooting:**

If you encounter any issues during the installation process, consult the Fedora documentation or seek help from the Fedora community.

Conclusion:

In this Way we successfully install the Fedora 42 Workstation on the Personal Computer.

Experiment No.02

Name of Experiment: Implement of different types of system calls.

Problem Statement: Implement the fork () and Write () system call in Linux operating System

Software /Hardware Requirements: GCC Compiler Linux Operating System

Theory:

A **system call** is a programmatic way in which a computer program requests a service from the kernel of the operating system on which it is executed. A system call is a way for programs to **interact with the operating system**. A computer program makes a system call when it requests the operating system's kernel. System call **provides** the services of the operating system to the user programs via the Application Program Interface (API). System calls are the only entry points into the kernel system and are executed in kernel mode.



- A user program can interact with the operating system using a system call. A number of services are requested by the program, and the OS responds by launching a number of systems calls to fulfill the request.
- A system call can be written in high-level languages like C or Pascal or in assembly language. If a high-level language is used, the operating system may directly invoke system calls, which are predefined functions.
- A system call is initiated by the program executing a specific instruction, which triggers a switch to kernel mode, allowing the program to request a service from the OS. The OS then handles the request, performs the necessary operations, and returns the result back to the program.
- System calls are essential for the proper functioning of an operating system, as they provide a standardized way for programs to access system resources. Without system calls, each program would need to implement its methods for accessing hardware and system services, leading to inconsistent and error-prone behavior.

Services Provided by System Calls

- Process Creation and Management
- Main Memory Management
- File Access, Directory, and File System Management
- Device Handling(I/O)
- Protection
- Networking, etc.
- **Process Control:** end, abort, create, terminate, allocate, and free memory.
- **File Management:** create, open, close, delete, read files, etc.

- **Device Management**
- **Information Maintenance**
- **Communication**

Features of System Calls

- **Interface:** System calls provide a well-defined interface between user programs and the operating system. Programs make requests by calling specific functions, and the operating system responds by executing the requested service and returning a result.
- **Protection:** System calls are used to access privileged operations that are not available to normal user programs. The operating system uses this privilege to protect the system from malicious or unauthorized access.
- **Kernel Mode:** When a system call is made, the program is temporarily switched from user mode to kernel mode. In kernel mode, the program has access to all system resources, including hardware, memory, and other processes.
- **Context Switching:** A system call requires a context switch, which involves saving the state of the current process and switching to the kernel mode to execute the requested service. This can introduce overhead, which can impact system performance.
- **Error Handling:** System calls can return error codes to indicate problems with the requested service. Programs must check for these errors and handle them appropriately.
- **Synchronization:** System calls can be used to synchronize access to shared resources, such as files or network connections. The operating system provides synchronization mechanisms, such as locks or semaphores, to ensure that multiple programs can access these resources safely.

How does System Call Work?

Here is a detailed explanation step by step how system calls work:

- **Users need special resources:** Sometimes programs need to do some special things that can't be done without the permission of the OS like reading from a file, writing to a file, getting any information from the hardware, or requesting a space in memory.
- **The program makes a system call request:** There are special predefined instructions to make a request to the operating system. These instructions are nothing but just a "system call". The program uses these system calls in its code when needed.
- **Operating system sees the system call:** When the OS sees the system call then it recognizes that the program needs help at this time so it temporarily stops the program execution and gives all the control to a special part of itself called 'Kernel'. Now 'Kernel' solves the need of the program.
- **The operating system performs the operations:** Now the operating system performs the operation that is requested by the program. Example: reading content from a file etc.
- **Operating system give control back to the program :** After performing the special operation, OS give control back to the program for further execution of program.

Examples of a System Call in Windows and Unix

System calls for Windows and Unix come in many different forms. These are listed in the table below as follows:

Process	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	Fork() Exit() Wait()
File manipulation	CreateFile() ReadFile() WriteFile()	Open() Read() Write() Close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	Ioctl() Read() Write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	Getpid() Alarm() Sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	Pipe() Shmget() Mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	Chmod() Umask() Chown()

Program Code:

```
/* C Program to Implement fork() System Call */
```

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    // Create a new process
```

```
    pid = fork();
```

```
    // Check for error
```

```
    if (pid < 0) {
```

```
        perror("Fork failed");
```

```
        return 1;
```

```
}

// Child process block
if (pid == 0) {
    printf("Child Process:\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID (from child): %d\n", getppid());
}
// Parent process block
else {
    printf("Parent Process:\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID (from parent): %d\n", pid);
}

return 0;
}
```

Output:

Compilation and Execution (Linux terminal)

1. **Save the file** as `fork_example.c`
2. **Compile the program** using GCC:

```
gcc fork_example.c -o fork_example
```

3. **Run the executable:**

```
./fork_example
```

```
Parent Process:
Parent PID: 12345
Child PID (from parent): 12346
Child Process:
Child PID: 12346
Parent PID (from child): 12345
```

/* C Program to Implement write() System Call */

```
#include <unistd.h>    // for write()
#include <fcntl.h>     // for open() and O_* constants
#include <sys/types.h>
#include <sys/stat.h>
#include <stdio.h>     // for perror()

int main() {
    int fd;
    const char *text = "Hello from write() system call in C!\n";

    // Open a file (or create it if it doesn't exist)
    fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
if (fd < 0) {
    perror("File open failed");
    return 1;
}

// Write to the file
ssize_t bytes_written = write(fd, text, 37); // write 37 bytes

if (bytes_written < 0) {
    perror("Write failed");
    close(fd);
    return 1;
}

printf("Successfully written %ld bytes to output.txt\n", bytes_written);

// Close the file
close(fd);

return 0;
}
```

- Save the code as `write_example.c`
- Compile with `gcc`:

```
gcc write_example.c -o write_example
```

- Run the program:

```
./write_example
```

- Check the contents of `output.txt`:

```
cat output.txt
```

Conclusion: In this way we implement `fork()` and `Write()` system call in operating system

Experiment No.03

Name of Experiment: Write a program to implement FCFS, SJF and priority based Algorithm.

Problem Statement: Implement a C program for FCFS SJF and Priority based Algorithm

Software /Hardware Requirements: Turbo C++ Compiler Personal Computer

Theory:

SJF Algorithm

Shortest Job First (SJF) is a non-preemptive scheduling algorithm that selects the process with the shortest burst time (execution time) to execute next. It is also known as Shortest Job Next (SJN).

Working Principle:

- At any scheduling point, the CPU is assigned to the process with the smallest burst time.
- If two processes have the same burst time, FCFS (First Come First Serve) is used as a tie-breaker.
- Once a process starts execution, it runs to completion (non-preemptive).

Steps:

1. Sort all processes by **burst time**.
2. Execute the process with the **shortest burst time**.
3. Calculate:
 - **Waiting Time (WT) = Start Time – Arrival Time**
 - **Turnaround Time (TAT) = Completion Time – Arrival Time**

FCFS Algorithm

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm, where the process that arrives first gets executed first. It follows the FIFO (First In, First Out) principle, like a queue.

- **Waiting Time (WT) = Start Time – Arrival Time**
- **Turnaround Time (TAT) = Completion Time – Arrival Time**
- **TAT = WT + Burst Time**

Working Principle:

1. Processes are added to the ready queue in the order they arrive.
2. CPU picks the first process in the queue and runs it until completion.
3. No preemption – once a process starts, it runs till it finishes.

Priority Based Algorithm

Priority Scheduling is a CPU scheduling algorithm where each process is assigned a priority, and the CPU is allocated to the process with the highest priority.

- Lower numerical value = Higher priority (by default).

- Can be preemptive or non-preemptive

Working Principle:

1. Each process has a priority number.
2. The CPU is assigned to the process with the highest priority.
3. If two processes have the same priority, **FCFS** is used.
4. In **pre-emptive mode**, a process with higher priority can interrupt a running process.

Program Code:

/ Write a program in C to Implement FCFS, SJF and Priority Scheduling Algorithm */*

```
#include <stdio.h>
struct Process {
    int pid;
    int burst;
    int priority;
    int waiting;
    int turnaround;
};

// Function to display results
void display(struct Process p[], int n, const char* algo_name) {
    float total_wait = 0, total_turnaround = 0;
    printf("\n--- %s Scheduling ---\n", algo_name);
    printf("PID\tBurst\tPriority\tWaiting\tTurnaround\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\t%d\n",
            p[i].pid, p[i].burst, p[i].priority,
            p[i].waiting, p[i].turnaround);

        total_wait += p[i].waiting;
        total_turnaround += p[i].turnaround;
    }

    printf("Average Waiting Time: %.2f\n", total_wait / n);
    printf("Average Turnaround Time: %.2f\n", total_turnaround / n);
}

// FCFS Scheduling
void fcfs(struct Process p[], int n) {
    p[0].waiting = 0;
    for (int i = 1; i < n; i++) {
        p[i].waiting = p[i - 1].waiting + p[i - 1].burst;
    }
    for (int i = 0; i < n; i++) {
```

```
        p[i].turnaround = p[i].waiting + p[i].burst;
    }
}

// SJF Scheduling (Non-preemptive)
void sjf(struct Process p[], int n) {
    struct Process temp;
    // Sort based on burst time
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (p[j].burst < p[i].burst) {
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }

    p[0].waiting = 0;
    for (int i = 1; i < n; i++)
        p[i].waiting = p[i - 1].waiting + p[i - 1].burst;

    for (int i = 0; i < n; i++)
        p[i].turnaround = p[i].waiting + p[i].burst;
}

// Priority Scheduling (Non-preemptive, lower value = higher
priority)
void priority(struct Process p[], int n) {
    struct Process temp;
    // Sort based on priority
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (p[j].priority < p[i].priority) {
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }

    p[0].waiting = 0;
    for (int i = 1; i < n; i++)
        p[i].waiting = p[i - 1].waiting + p[i - 1].burst;

    for (int i = 0; i < n; i++)
        p[i].turnaround = p[i].waiting + p[i].burst;
}

int main() {
    int n;
    struct Process p[10], fcfs_arr[10], sjf_arr[10], prio_arr[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
```

```
p[i].pid = i + 1;
printf("Enter Burst Time and Priority for Process %d: ",
p[i].pid);
scanf("%d%d", &p[i].burst, &p[i].priority);
}

// Copy data to preserve original for each algorithm
for (int i = 0; i < n; i++) {
    fcfs_arr[i] = p[i];
    sjf_arr[i] = p[i];
    prio_arr[i] = p[i];
}

fcfs(fcfs_arr, n);
display(fcfs_arr, n, "FCFS");

sjf(sjf_arr, n);
display(sjf_arr, n, "SJF");

priority(prio_arr, n);
display(prio_arr, n, "Priority");

return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Burst Time and Priority for Process 1: 6 3
Enter Burst Time and Priority for Process 2: 8 1
Enter Burst Time and Priority for Process 3: 7 2
```

--- FCFS Scheduling ---

PID	Burst	Priority	Waiting	Turnaround
1	6	3	0	6
2	8	1	6	14
3	7	2	14	21

Average Waiting Time: 6.67
Average Turnaround Time: 13.67

--- SJF Scheduling ---

PID	Burst	Priority	Waiting	Turnaround
1	6	3	0	6
3	7	2	6	13
2	8	1	13	21

Average Waiting Time: 6.33
Average Turnaround Time: 13.33

--- Priority Scheduling ---

PID	Burst	Priority	Waiting	Turnaround
2	8	1	0	8
3	7	2	8	15
1	6	3	15	21

Average Waiting Time: 7.67

Average Turnaround Time: 14.67

Conclusion:

In this way we implement a program to implement FCFS, SJF and priority based Algorithm

Experiment No.04

Name of Experiment: Write a program to implement Round Robin Algorithm.

Problem Statement: Write a program in C to implement Round Robin Scheduling algorithms

Software /Hardware Requirements: Turbo C++ Compiler, Personal Computer

Theory:

Round Robin (RR) is a preemptive CPU scheduling algorithm where each process is assigned a fixed time slot (Time Quantum) in a cyclic manner.

It is one of the simplest and most widely used scheduling techniques, especially in time-sharing systems.

Working Principle:

1. All processes are kept in a **ready queue**.
2. CPU picks the first process and gives it the **CPU for a fixed time quantum**.
3. If the process **finishes within the time**, it is removed from the queue.
4. If it **does not finish**, it is preempted and placed at the **back of the queue** with its **remaining**
5. **burst time**.
6. This cycle repeats until all processes are completed.

Key Terms:

- **Time Quantum (TQ):**
The fixed time period a process can execute before being preempted.
- **Context Switching:**
Saving and loading the state of a process when switching between processes.

Formulas:

- Turnaround Time (TAT) = Completion Time – Arrival Time
- Waiting Time (WT) = Turnaround Time – Burst Time

Program Code:

/* Write a program in C to implement Round Robin Scheduling Algorithm*/

```
#include <stdio.h>
struct Process {
    int pid;
    int arrival;
    int burst;
    int remaining;
    int waiting;
    int turnaround;
};
```

```
int main() {
    int n, time = 0, quantum, completed = 0;
    struct Process p[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input arrival and burst times
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter Arrival Time and Burst Time for Process %d: ",
p[i].pid);
        scanf("%d%d", &p[i].arrival, &p[i].burst);
        p[i].remaining = p[i].burst;
        p[i].waiting = 0;
        p[i].turnaround = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &quantum);

    int done;
    do {
        done = 1;
        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0 && p[i].arrival <= time) {
                done = 0;
                if (p[i].remaining > quantum) {
                    time += quantum;
                    p[i].remaining -= quantum;
                } else {
                    time += p[i].remaining;
                    p[i].waiting = time - p[i].arrival - p[i].burst;
                    p[i].turnaround = time - p[i].arrival;
                    p[i].remaining = 0;
                    completed++;
                }
            } else if (p[i].arrival > time && p[i].remaining > 0) {
                // If no process has arrived, skip time forward
                time++;
                break;
            }
        }
    } while (completed < n);

    // Output results
    float total_wait = 0, total_turn = 0;
    printf("\nPID\tArrival\tBurst\tWaiting\tTurnaround\n");
    for (int i = 0; i < n; i++) {
```

```
printf("%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].arrival, p[i].burst, p[i].waiting,
p[i].turnaround);
    total_wait += p[i].waiting;
    total_turn += p[i].turnaround;
}

printf("Average Waiting Time: %.2f\n", total_wait / n);
printf("Average Turnaround Time: %.2f\n", total_turn / n);

return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time for Process 1: 0 5
Enter Arrival Time and Burst Time for Process 2: 1 3
Enter Arrival Time and Burst Time for Process 3: 2 1
Enter Time Quantum: 2
```

PID	Arrival	Burst	Waiting	Turnaround
1	0	5	4	9
2	1	3	4	7
3	2	1	2	3

Average Waiting Time: 3.33

Average Turnaround Time: 6.33

Conclusion: In this way we implement a program in C for Round Robin Scheduling algorithms.

Experiment No.05

Name of Experiment: Write a program to implement Producer Consumer Problem

Problem Statement: Write a program in C to implement producer Consumer Problem

Software /Hardware Requirements: GCC Compiler, Personal Computer

Theory:

The Producer-Consumer problem is a classic synchronization issue in operating systems. It involves two types of processes: producers, which generate data, and consumers, which process that data. Both share a common buffer. The challenge is to ensure that the producer doesn't add data to a full buffer and the consumer doesn't remove data from an empty buffer while avoiding conflicts when accessing the buffer. In this article, we are going to solve this problem by using semaphores.

What is a Semaphore?

A semaphore is a synchronization tool used in computing to manage access to shared resources. It works like a signal that allows multiple processes or threads to coordinate their actions. Semaphores use counters to keep track of how many resources are available, ensuring that no two processes can use the same resource at the same time, thus preventing conflicts and ensuring orderly execution.

A **semaphore** *S* is an integer variable that can be accessed only through two standard operations: `wait()` and `signal()`. The `wait()` operation reduces the value of the semaphore by 1 and the `signal()` operation increases its value by 1.

```
wait(S){
while(S<=0);    // busy waiting
S--;
}

signal(S){
S++;
}
```

Types of Semaphore

- **Binary Semaphore** - This is similar to mutex lock but not the same thing. It can have only two values - 0 and 1. Its value is initialized to 1. It is used to implement the solution of critical section problems with multiple processes.
- **Counting Semaphore** - Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

Producer Consumer Problem Statement

We have a buffer of fixed size. A producer can produce an item and can place it in the buffer. A consumer can pick items and consume them. We need to ensure that when a producer is placing an item in the buffer, then at the same time consumer should not consume any item. In this problem, the buffer is the critical section.

To solve this problem, we need two counting semaphores - Full and Empty. "Full" keeps track of some items in the buffer at any given time and "Empty" keeps track of many unoccupied slots.

Initialization of semaphores

mutex = 1

Full = 0 // Initially, all slots are empty. Thus full slots are 0

Empty = n // All slots are empty initially

Solution for Producer

```
do{  
  
//produce an item  
  
wait(empty);  
wait(mutex);  
  
//place in buffer  
  
signal(mutex);  
signal(full);  
  
}while(true)
```

When producer produces an item then the value of "empty" is reduced by 1 because one slot will be filled now. The value of mutex is also reduced to prevent consumer to access the buffer. Now, the producer has placed the item and thus the value of "full" is increased by 1. The value of mutex is also increased by 1 because the task of producer has been completed and consumer can access the buffer.

Solution for Consumer

```
do{  
  
wait(full);  
wait(mutex);  
  
// consume item from buffer  
  
signal(mutex);  
signal(empty);  
  
}while(true)
```

As the consumer is removing an item from buffer, therefore the value of "full" is reduced by 1 and the value of mutex is also reduced so that the producer cannot access the buffer at this moment. Now, the consumer has consumed the item, thus increasing the value of "empty" by 1. The value of mutex is also increased so that producer can access the buffer now.

Program Code:

/* Write a program in C for producer-Consumer problem */

```
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
#include <semaphore.h>  
#include <unistd.h>  
  
#define SIZE 5 // Size of the buffer
```

```
int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

// Producer function
void* producer(void* arg) {
    int item;
    for (int i = 1; i <= 10; i++) {
        item = rand() % 100;

        sem_wait(&empty);          // Wait for empty slot
        pthread_mutex_lock(&mutex); // Enter critical section

        buffer[in] = item;
        printf("Producer produced: %d at %d\n", item, in);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex); // Exit critical section
        sem_post(&full);              // Signal that buffer has
data
        sleep(1);
    }
    return NULL;
}

// Consumer function
void* consumer(void* arg) {
    int item;
    for (int i = 1; i <= 10; i++) {
        sem_wait(&full);          // Wait for filled slot
        pthread_mutex_lock(&mutex); // Enter critical section

        item = buffer[out];
        printf("\tConsumer consumed: %d from %d\n", item, out);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex); // Exit critical section
        sem_post(&empty);          // Signal that buffer has
space
        sleep(2);
    }
    return NULL;
}

int main() {
```

```
pthread_t prod, cons;

// Initialize semaphores and mutex
sem_init(&empty, 0, SIZE);
sem_init(&full, 0, 0);
pthread_mutex_init(&mutex, NULL);

// Create producer and consumer threads
pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);

// Wait for threads to finish
pthread_join(prod, NULL);
pthread_join(cons, NULL);

// Destroy semaphores and mutex
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

return 0;
}
```

Output:

```
gcc producer_consumer.c -o pc -lpthread
./pc
Producer produced: 35 at 0
  Consumer consumed: 35 from 0
Producer produced: 57 at 1
Producer produced: 18 at 2
  Consumer consumed: 57 from 1
Producer produced: 99 at 3
  Consumer consumed: 18 from 2
Producer produced: 12 at 4
  Consumer consumed: 99 from 3
...
```

Conclusion:

In this way we successfully compiled and executed a program for producer –consumer problem

Experiment No.06

Name of Experiment: Write a program to implement Dining-Philosophers Problem

Problem Statement:

The Dining Philosophers Problem is a classic synchronization and concurrency problem that illustrates challenges in resource allocation, deadlock avoidance, and mutual exclusion in multi-threaded or multi-process systems.

Five philosophers are seated around a circular dining table. Each philosopher alternates between **thinking** and **eating**. Between every two philosophers is a single **fork**, and a philosopher requires **both the left and right forks** to eat.

The goal is to design a system where:

- No two philosophers eat using the same fork at the same time.
- Philosophers do not starve (i.e., every philosopher eventually gets a chance to eat).
- The system avoids **deadlock**, **starvation**, and **race conditions**.

Software /Hardware Requirements: GCC compiler, Personal Computer, Fedora Workstation42

Theory:

The **Dining Philosopher Problem** is a classic synchronization and concurrency problem that deals with resource sharing, deadlock, and starvation in systems where multiple processes require limited resources. In this article, we will discuss the Dining Philosopher Problem in detail along with proper implementation.

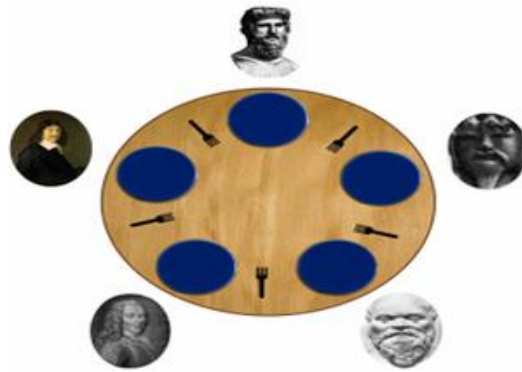
Problem Statement

The **Dining Philosopher Problem** involves 'n' philosophers sitting around a circular table. Each philosopher alternates between two states: **thinking** and **eating**. To eat, a philosopher needs two chopsticks, one on their left and one on their right. However, the number of chopsticks is equal to the number of philosophers, and each chopstick is shared between two neighboring philosophers.

The standard problem considers the value of 'n' as 5 i.e. we deal with 5 Philosophers sitting around a circular table.

Constraints and Conditions for the Problem

- Every Philosopher needs two forks to eat.
- Every Philosopher may pick up the forks on the left or right but only one fork at once.
- Philosophers only eat when they have two forks. We have to design such a protocol i.e. pre and post protocol which ensures that a philosopher only eats if he or she has two forks.
- Each fork is either clean or dirty.



Solution

The correctness properties it needs to satisfy are:

- **Mutual Exclusion Principle:** No two Philosophers can have the two forks simultaneously.
- **Free from Deadlock:** Each philosopher can get the chance to eat in a certain finite time.
- **Free from Starvation:** When few Philosophers are waiting then one gets a chance to eat in a while.
- **No strict Alternation**
- **Proper utilization of time**

Program Code:

/ Write a program for Implementing dining-Philosophers problem */*

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_t philosophers[N];
pthread_mutex_t forks[N];

void* philosopher(void* num) {
    int id = *(int*)num;

    while (1) {
        // Thinking
        printf("Philosopher %d is thinking.\n", id);
        sleep(rand() % 3);

        // Pick up forks (deadlock-free: odd picks left first, even
        picks right first)
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[(id + 1) % N]); // right fork
            pthread_mutex_lock(&forks[id]); // left fork
        } else {
            pthread_mutex_lock(&forks[id]); // left fork
```

```
        pthread_mutex_lock(&forks[(id + 1) % N]); // right fork
    }

    // Eating
    printf("Philosopher %d is eating.\n", id);
    sleep(rand() % 2 + 1);

    // Put down forks
    pthread_mutex_unlock(&forks[id]);
    pthread_mutex_unlock(&forks[(id + 1) % N]);

    printf("Philosopher %d has finished eating and put down
forks.\n", id);
}

return NULL;
}

int main() {
    int i;
    int ids[N];

    // Initialize mutexes
    for (i = 0; i < N; i++)
        pthread_mutex_init(&forks[i], NULL);

    // Create philosopher threads
    for (i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher,
&ids[i]);
    }

    // Join philosopher threads (infinite loop; use Ctrl+C to stop)
    for (i = 0; i < N; i++)
        pthread_join(philosophers[i], NULL);

    // Destroy mutexes
    for (i = 0; i < N; i++)
        pthread_mutex_destroy(&forks[i]);

    return 0;
}
```

Output:

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
```

Philosopher 2 is eating.

Philosopher 4 is eating.

Philosopher 2 has finished eating and put down forks.

Philosopher 4 has finished eating and put down forks.

...

Conclusion:

In this we successfully compiled and executed a program in C for dining-philosopher problem.

Experiment No.07

Name of Experiment: Write a program to implement Bankers algorithm

Problem Statement:

In multiprogramming environments, multiple processes may request access to limited system resources (like CPU, memory, I/O devices). If resource allocation is not handled carefully, it may lead to **deadlock**, where a set of processes wait indefinitely for resources held by each other.

The **Banker's Algorithm**, proposed by Edsger Dijkstra, is a **deadlock avoidance algorithm** that simulates the allocation of predetermined maximum possible resources to ensure that the system remains in a **safe state**.

The objective is to implement a system that:

- Decides whether or not to grant resource requests based on system safety.
- Avoids unsafe states and prevents deadlocks.
- Tracks the available, maximum, allocated, and needed resources for each process.

Objectives:

Implement the Banker's algorithm to:

1. Check whether the system is in a safe state.
2. Grant or deny resource requests from processes based on safety check.
3. Ensure all processes can complete without entering an unsafe (deadlock-prone) state.

Software /Hardware Requirements: GCC Compiler, Linux OS, Personal Computer

Theory:

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks. The Banker's Algorithm is a smart way for computer systems to manage how programs use resources, like memory or CPU time. It helps prevent situations where programs get stuck and can not finish their tasks. This condition is known as deadlock. By keeping track of what resources each program needs and what's available, the banker algorithm makes sure that programs only get what they need in a safe order. Why Banker's Algorithm is Named So? The banker's algorithm is named so because it is used in the banking system to check whether a loan can be sanctioned to a person or not. Suppose there are n number of account holders in a bank and the total sum of their money is S . Let us assume that the bank has a certain amount of money Y . If a person applies for a loan then the bank first subtracts the loan amount from the total money that the bank has (Y) and if the remaining amount is greater than S then only the loan is sanctioned. It is done because if all the account holders come to withdraw their money then the bank can easily do it.

It also helps the OS to successfully share the resources between all the processes. The banker's algorithm uses the notation of a safe allocation state to ensure that granting a resource request cannot lead to a deadlock either immediately or in the future. In other words, the bank would never allocate

its money in such a way that it can no longer satisfy the needs of all its customers. The bank would try to be in a safe state always.

Components of the Banker's Algorithm

The following **Data structures** are used to implement the Banker's Algorithm:

Let '**n**' be the number of processes in the system and '**m**' be the number of resource types.

1. Available

It is a 1-D array of size '**m**' indicating the number of available resources of each type.

Available[*j*] = *k* means there are '**k**' instances of resource type **R_j**

2. Max

It is a 2-d array of size '**n*m**' that defines the maximum demand of each process in a system.

Max[*i*, *j*] = *k* means process **P_i** may request at most '**k**' instances of resource type **R_j**.

3. Allocation

It is a 2-d array of size '**n*m**' that defines the number of resources of each type currently allocated to each process.

Allocation[*i*, *j*] = *k* means process **P_i** is currently allocated '**k**' instances of resource type **R_j**

4. Need

It is a 2-d array of size '**n*m**' that indicates the remaining resource need of each process.

Need [*i*, *j*] = *k* means process **P_i** currently needs '**k**' instances of resource type **R_j**

Need [*i*, *j*] = Max [*i*, *j*] – Allocation [*i*, *j*]

Program Code:

```
/* Implementing the Banker's Algorithm in C*/
```

```
#include <stdio.h>
#include <stdbool.h>
#define MAX_P 10      // maximum number of processes
#define MAX_R 10      // maximum number of resource types

int n, m;              // n = processes, m = resources
int Max[MAX_P][MAX_R];
int Allocation[MAX_P][MAX_R];
int Need[MAX_P][MAX_R];
int Available[MAX_R];

/* ----- utility ----- */
void calc_need(void)
{
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j)
            Need[i][j] = Max[i][j] - Allocation[i][j];
}

void print_state(void)
{
    puts("\nCurrent State:");
    puts("Process | Allocation |    Max    |    Need    ");
```

```
puts("-----");
for (int i = 0; i < n; ++i) {
    printf("P%-6d ", i);
    for (int j = 0; j < m; ++j) printf("%3d", Allocation[i][j]);
    printf("      ");
    for (int j = 0; j < m; ++j) printf("%3d", Max[i][j]);
    printf("      ");
    for (int j = 0; j < m; ++j) printf("%3d", Need[i][j]);
    putchar('\n');
}
printf("Available:");
for (int j = 0; j < m; ++j) printf(" %3d", Available[j]);
puts("\n");
}

/* ----- safety algorithm ----- */
bool is_safe(int safe_seq[MAX_P])
{
    int work[MAX_R];
    bool finish[MAX_P] = {false};

    for (int j = 0; j < m; ++j) work[j] = Available[j];

    int count = 0;
    while (count < n) {
        bool found = false;
        for (int i = 0; i < n; ++i) {
            if (!finish[i]) {
                bool can_run = true;
                for (int j = 0; j < m; ++j)
                    if (Need[i][j] > work[j]) { can_run = false;
break; }

                if (can_run) {
                    for (int j = 0; j < m; ++j) work[j] +=
Allocation[i][j];
                    safe_seq[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }
        if (!found) return false; // no suitable process found =>
unsafe
    }
    return true;
}

/* ----- resource-request algorithm ----- */
```

```
bool request_resources(int p, int req[])
{
    /* Step-1: request ≤ need ? */
    for (int j = 0; j < m; ++j)
        if (req[j] > Need[p][j]) return false;

    /* Step-2: request ≤ available ? */
    for (int j = 0; j < m; ++j)
        if (req[j] > Available[j]) return false;    // must wait

    /* Step-3: pretend to allocate */
    for (int j = 0; j < m; ++j) {
        Available[j]      -= req[j];
        Allocation[p][j] += req[j];
        Need[p][j]        -= req[j];
    }

    int seq[MAX_P];
    bool safe = is_safe(seq);

    if (!safe) {          // rollback
        for (int j = 0; j < m; ++j) {
            Available[j]      += req[j];
            Allocation[p][j] -= req[j];
            Need[p][j]        += req[j];
        }
    }
    return safe;
}

/* ----- main driver ----- */
int main(void)
{
    printf("Enter number of processes (≤%d): ", MAX_P); scanf("%d",
    &n);
    printf("Enter number of resource types (≤%d): ", MAX_R);
    scanf("%d", &m);

    puts("\nEnter Allocation matrix:");
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j)
            scanf("%d", &Allocation[i][j]);

    puts("Enter Max matrix:");
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j)
            scanf("%d", &Max[i][j]);

    puts("Enter Available vector:");
```

```
for (int j = 0; j < m; ++j) scanf("%d", &Available[j]);

calc_need();

while (1) {
    printf("\nMenu:\n"
           "1. Display State & Check Safety\n"
           "2. Make a Resource Request\n"
           "0. Exit\nChoice: ");
    int ch; scanf("%d", &ch);

    if (ch == 0) break;

    if (ch == 1) {
        print_state();
        int seq[MAX_P];
        if (is_safe(seq)) {
            printf("System is in a SAFE state.\nSafe sequence: ");
            for (int i = 0; i < n; ++i) printf("P%d%s", seq[i],
(i==n-1?"\n":" → "));
        } else {
            puts("System is in an UNSAFE state!");
        }
    }
    else if (ch == 2) {
        int p; printf("Requesting process number (0-%d): ", n-1);
scanf("%d", &p);
        int req[MAX_R];
        printf("Enter request vector of size %d: ", m);
        for (int j = 0; j < m; ++j) scanf("%d", &req[j]);

        if (request_resources(p, req)) {
            puts("\nRequest GRANTED. New state:");
        } else {
            puts("\nRequest DENIED (would lead to unsafe
state).");
        }
        print_state();
    }
}
return 0;
}
```

Output:

```
Enter number of processes (≤10): 5
Enter number of resource types (≤10): 3

Enter Allocation matrix:
0 1 0   2 0 0   3 0 2   2 1 1   0 0 2
Enter Max matrix:
```

7 5 3 3 2 2 9 0 2 2 2 2 4 3 3

Enter Available vector:

3 3 2

Menu:

1. Display State & Check Safety

2. Make a Resource Request

0. Exit

Choice: 1

Current State:

Process	Allocation	Max	Need
---------	------------	-----	------

P0	0 1 0	7 5 3	7 4 3
----	-------	-------	-------

P1	2 0 0	3 2 2	1 2 2
----	-------	-------	-------

P2	3 0 2	9 0 2	6 0 0
----	-------	-------	-------

P3	2 1 1	2 2 2	0 1 1
----	-------	-------	-------

P4	0 0 2	4 3 3	4 3 1
----	-------	-------	-------

Available: 3 3 2

System is in a **SAFE** state.

Safe sequence: P1 → P3 → P4 → P0 → P2

Conclusion: In this way we successfully compiled and executed a program for Banker's Algorithm

Experiment No.08

Name of Experiment: Write a program to implement Page Replacement

Problem Statement:

In a virtual memory system, when a running process requests a page that is not present in the main memory (a page fault), the operating system must decide which existing page to replace. This decision is critical to system performance. The goal is to implement the **FIFO page replacement algorithm**, where the oldest loaded page in memory is the first one to be replaced when a new page needs to be loaded and memory is full.

The program should:

- Accept a page reference string and the number of available frames.
- Simulate the FIFO page replacement process.
- Track and display page hits and page faults.
- Print the frame status after each page request.

Expected Outcome:

A clear visualization of how pages are loaded and replaced.

Total number of page hits and page faults.

Better understanding of how FIFO operates in memory management.

Software /Hardware Requirements: GCC Compiler, Linux OS, PC

Theory:

In modern operating systems, memory management is crucial. When there is not enough physical memory to hold all the pages required by processes, the system uses a **page replacement algorithm** to decide which page to remove from memory.

FIFO (First-In-First-Out) is one of the simplest page replacement algorithms.

What is FIFO Page Replacement?

FIFO (First-In-First-Out) page replacement algorithm works on the principle of replacing the **oldest page** in memory – the one that entered first – when a new page needs to be loaded and there are no free frames available.

Key Features:

- **Simple and Easy to Implement**
- Maintains pages in a **queue** (circular or linear)
- The **front** of the queue holds the **oldest page**, and the **rear** holds the most recent one.

- On a **page fault**, the page at the front is removed, and the new page is added at the rear.

Working of FIFO Algorithm:

1. Start with an empty memory of fixed size (number of frames).
2. As each page from the reference string is processed:
 - If it is already in memory (a **hit**), do nothing.
 - If not (a **page fault**), and there is space, load it.
 - If memory is full, **remove the oldest page** (first inserted), and insert the new page.

Program Code:

```
#include <stdio.h>
int main() {
    int frames, pages[50], temp[50];
    int i, j, k, pageFaults = 0, pageHits = 0;
    int totalPages, pos = 0, flag, isHit;

    // Input number of pages
    printf("Enter total number of pages: ");
    scanf("%d", &totalPages);

    printf("Enter the page reference string:\n");
    for(i = 0; i < totalPages; i++) {
        scanf("%d", &pages[i]);
    }

    // Input number of frames
    printf("Enter number of frames: ");
    scanf("%d", &frames);

    for(i = 0; i < frames; i++) {
        temp[i] = -1; // Initialize all frames to -1
    }

    printf("\nPage\tFrames\t\tHit/Miss\n");

    for(i = 0; i < totalPages; i++) {
        isHit = 0;

        // Check if page is already in frame (Hit)
        for(j = 0; j < frames; j++) {
            if(temp[j] == pages[i]) {
                isHit = 1;
                pageHits++;
            }
        }
    }
}
```

```
        break;
    }
}

if(!isHit) {
    temp[pos] = pages[i]; // Replace the oldest page (FIFO)
    pos = (pos + 1) % frames;
    pageFaults++;
}

printf("%d\t", pages[i]);
for(k = 0; k < frames; k++) {
    if(temp[k] != -1)
        printf("%d ", temp[k]);
    else
        printf("- ");
}

if(isHit)
    printf("\t\tHit\n");
else
    printf("\t\tMiss\n");
}

printf("\nTotal Page Faults: %d\n", pageFaults);
printf("Total Page Hits: %d\n", pageHits);

return 0;
}
```

Output:

```
Enter total number of pages: 12
Enter the page reference string:
1 3 0 3 5 6 3 0 1 3 6 2
Enter number of frames: 3
```

Page	Frames	Hit/Miss
1	1 - -	Miss
3	1 3 -	Miss
0	1 3 0	Miss
3	1 3 0	Hit
5	5 3 0	Miss
6	5 6 0	Miss
3	5 6 3	Miss
0	0 6 3	Miss
1	0 1 3	Miss
3	0 1 3	Hit
6	6 1 3	Miss
2	6 2 3	Miss

Total Page Faults: 10

Total Page Hits: 2

Conclusion: In this way we successfully compile and executed a program for FIFO page Replacement algorithm

Experiment No.09

Name of Experiment: Write a program to implement Memory management algorithm

Problem Statement:

In multiprogramming operating systems, memory is divided into fixed-size or variable-size blocks and allocated to processes based on their size requirements. Efficient memory allocation is essential to minimize internal and external fragmentation.

The objective is to implement the **First Fit Memory Allocation Algorithm**, where each process is allocated to the **first available memory block** that is large enough to accommodate it. This approach should simulate the allocation of memory blocks to processes and display whether the allocation was successful or not for each process.

The program should:

- Take the sizes of memory blocks and process sizes as input.
- Apply the **First Fit allocation strategy**.
- Display the allocation result showing which block is assigned to each process.
- Show unallocated processes if no suitable block is found.

Expected Outcome:

- A working simulation of the First Fit algorithm.
- A table showing process number, size, and the block number allocated (or not allocated).
- Understanding of how memory is allocated in real-time systems using First Fit.

Software /Hardware Requirements: GCC Compiler, Linux OS, PC

Theory:

In operating systems, memory management is a crucial function to allocate space to various processes efficiently. One of the most commonly used memory allocation strategies is the **First Fit Algorithm**, which is part of **contiguous memory allocation** methods.

What is First Fit Algorithm?

First Fit is a memory allocation algorithm where the **first available memory block** that is **large enough** to accommodate a process is selected for allocation.

This algorithm **scans memory from the beginning** and allocates the first block that satisfies the size requirement of the process.

Working of First Fit Algorithm:

1. Memory is divided into blocks of various sizes.
2. When a process arrives and needs memory:

- The system scans memory blocks from the beginning.
- The process is placed in the **first** free block large enough to hold it.
- If no such block is found, the process is not allocated.

Example:

Let's say the memory blocks are:
[100, 500, 200, 300, 600]

And process sizes are:
[212, 417, 112, 426]

Allocation (First Fit):

Process	Size	Block Allocated
P1	212	Block 2 (500)
P2	417	Block 5 (600)
P3	112	Block 3 (200)
P4	426	Not Allocated

Advantages:

- **Simple and fast** implementation.
- Requires **less time** to find a suitable block.
- **Good for real-time systems** where quick allocation is needed.

Disadvantages:

- Can cause **external fragmentation** (many small unallocated spaces).
- Does **not consider best utilization** of memory.
- Large blocks may be broken inefficiently by small processes.

Program Code:

```
#include <stdio.h>
int main() {
    int blockSize[10], processSize[10];
    int blockAllocated[10];
    int i, j, numBlocks, numProcesses;

    // Input number of blocks and their sizes
    printf("Enter number of memory blocks: ");
    scanf("%d", &numBlocks);
    printf("Enter sizes of %d blocks:\n", numBlocks);
```

```
for(i = 0; i < numBlocks; i++) {
    scanf("%d", &blockSize[i]);
}

// Input number of processes and their sizes
printf("Enter number of processes: ");
scanf("%d", &numProcesses);
printf("Enter sizes of %d processes:\n", numProcesses);
for(i = 0; i < numProcesses; i++) {
    scanf("%d", &processSize[i]);
}

// Initialize block allocation tracker
for(i = 0; i < numBlocks; i++) {
    blockAllocated[i] = 0;
}

// First Fit Allocation
printf("\nProcess No.\tProcess Size\tBlock No.\n");
for(i = 0; i < numProcesses; i++) {
    int allocated = 0;
    for(j = 0; j < numBlocks; j++) {
        if(blockAllocated[j] == 0 && blockSize[j] >= processSize[i])
        {
            // Allocate block j to process i
            printf("%d\t%d\t%d\n", i+1, processSize[i], j+1);
            blockSize[j] -= processSize[i]; // Reduce available size
            blockAllocated[j] = 1;          // Mark block as allocated
            allocated = 1;
            break;
        }
    }
    if(!allocated) {
        printf("%d\t%d\tNot Allocated\n", i+1,
processSize[i]);
    }
}

return 0;
}
```

Output:

```
Enter number of memory blocks: 5
Enter sizes of 5 blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426
Process No.      Process Size      Block No.
```

1	212	2
2	417	5
3	112	3
4	426	Not Allocated

Conclusion: In this way we successfully compiled and executed a program for First Fit Memory management Algorithm

Experiment No.10

Name of Experiment: Write a program to implement Disk Scheduling algorithms

Problem Statement: In computer operating systems, disk scheduling is used to determine the order in which disk I/O requests are processed. When multiple processes request access to the disk, the system must schedule these requests to optimize performance metrics like **seek time**, **latency**, and **throughput**.

The objective is to implement and compare different **disk scheduling algorithms** such as:

- **FCFS (First-Come-First-Serve)**
- **SSTF (Shortest Seek Time First)**
- **SCAN (Elevator Algorithm)**

The program should:

- Take a sequence of disk I/O requests and the initial position of the disk head as input.
- Simulate each algorithm's execution.
- Calculate and display the total **seeks time** and the order in which requests are served.
- Provide a way to compare algorithm performance for a given input.

Expected Output:

- The order of disk access for each algorithm.
- Total seek time taken by the disk head for each method.
- A comparison of efficiency among the algorithms.

Software /Hardware Requirements: GCC Compiler, Linux OS, PC

Theory:

• **FCFS (First-Come-First-Serve):**

- Services requests in the **order they arrive**.
- Simple and fair, but not optimal in terms of performance.

Pros: Easy to implement

Cons: May cause long seek times if requests are far apart

SSTF (Shortest Seek Time First):

- Selects the request **closest** to the current head position.

- Reduces seek time compared to FCFS.

Pros: Better performance than FCFS

Cons: May cause **starvation** of distant requests

SCAN (Elevator Algorithm):

- Head moves in one direction servicing all requests, then **reverses**.
- Like an elevator going up and down.

Pros: More uniform waiting time

Cons: Higher seek time than SSTF in some cases

C-SCAN (Circular SCAN):

- Head moves in one direction servicing requests, then jumps to the start.
- Provides a more **uniform wait time** than SCAN.

. LOOK and C-LOOK:

- Similar to SCAN and C-SCAN, but head only goes as far as the **last request** in that direction.
- Avoids unnecessary traversal of unused tracks.

Program Code:

```
/* Write a program to implement Disk scheduling Algorihtm*/
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
void FCFS(int requests[], int n, int head) {
    int seek = 0;
    printf("\nFCFS Disk Scheduling:\nOrder: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", requests[i]);
        seek += abs(requests[i] - head);
        head = requests[i];
    }
    printf("\nTotal Seek Time: %d\n", seek);
}

void SSTF(int requests[], int n, int head) {
    int seek = 0, completed = 0;
    int visited[100] = {0};

    printf("\nSSTF Disk Scheduling:\nOrder: ");

    while (completed < n) {
        int min = 10000, index = -1;
```

```
        for (int i = 0; i < n; i++) {
            if (!visited[i] && abs(requests[i] - head) < min) {
                min = abs(requests[i] - head);
                index = i;
            }
        }
        visited[index] = 1;
        printf("%d ", requests[index]);
        seek += abs(requests[index] - head);
        head = requests[index];
        completed++;
    }

    printf("\nTotal Seek Time: %d\n", seek);
}

void SCAN(int requests[], int n, int head, int disk_size) {
    int seek = 0;
    int temp[100], index = 0;

    // Separate and sort
    for (int i = 0; i < n; i++)
        temp[i] = requests[i];
    temp[n++] = 0;
    temp[n++] = disk_size - 1;

    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (temp[i] > temp[j]) {
                int t = temp[i];
                temp[i] = temp[j];
                temp[j] = t;
            }
        }
    }

    // Find starting index
    for (int i = 0; i < n; i++) {
        if (temp[i] >= head) {
            index = i;
            break;
        }
    }

    printf("\nSCAN Disk Scheduling (Head moving right then left):\nOrder: ");
    for (int i = index; i < n; i++) {
        printf("%d ", temp[i]);
        seek += abs(temp[i] - head);
    }
}
```

```
        head = temp[i];
    }
    for (int i = index - 1; i >= 0; i--) {
        printf("%d ", temp[i]);
        seek += abs(temp[i] - head);
        head = temp[i];
    }

    printf("\nTotal Seek Time: %d\n", seek);
}

int main() {
    int requests[100], n, head, choice, disk_size;

    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the request sequence:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &requests[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter total disk size (e.g., 200): ");
    scanf("%d", &disk_size);

    do {
        printf("\nDisk Scheduling Algorithms:\n");
        printf("1. FCFS\n2. SSTF\n3. SCAN\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                FCFS(requests, n, head);
                break;
            case 2:
                SSTF(requests, n, head);
                break;
            case 3:
                SCAN(requests, n, head, disk_size);
                break;
            case 4:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice. Try again.\n");
        }
    } while (choice != 4);
}
```

```
    return 0;  
}
```

Output:

```
Enter number of disk requests: 8  
Enter the request sequence:  
98 183 37 122 14 124 65 67  
Enter initial head position: 53  
Enter total disk size (e.g., 200): 200
```

FCFS Disk Scheduling:

```
Order: 98 183 37 122 14 124 65 67  
Total Seek Time: 640
```

SSTF Disk Scheduling:

```
Order: 65 67 37 14 98 122 124 183  
Total Seek Time: 236
```

SCAN Disk Scheduling:

```
Order: 65 67 98 122 124 183 199 37 14 0  
Total Seek Time: 382
```

Conclusion: In this way we successfully implemented a program for disk scheduling algorithm